



Lesson 6.3 — PPR (Partial Prerendering)

Now we get to the **really interesting part**.

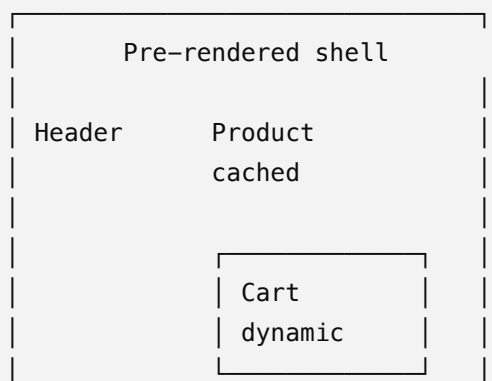
You've already learned the old mental model:

```
Route
↓
Is it static or dynamic?
↓
SSG / ISR / SSR
```



With **Cache Components / PPR**, the mental model becomes more granular:

```
Route
↓
Which parts can be prepared ahead of time?
Which parts need request-time data?
↓
```



↓
Dynamic part streamed
when request happens

1. What is PPR?

PPR = Partial Prerendering.

It means:

Pre-render the parts of a page that can be prepared ahead of time, while leaving dynamic parts to be resolved at request time.

Imagine:

/product/123



Your page contains:

```
<ProductPage>
  <Header />
  <Product />
  <Recommendations />
  <Cart />
</ProductPage>
```

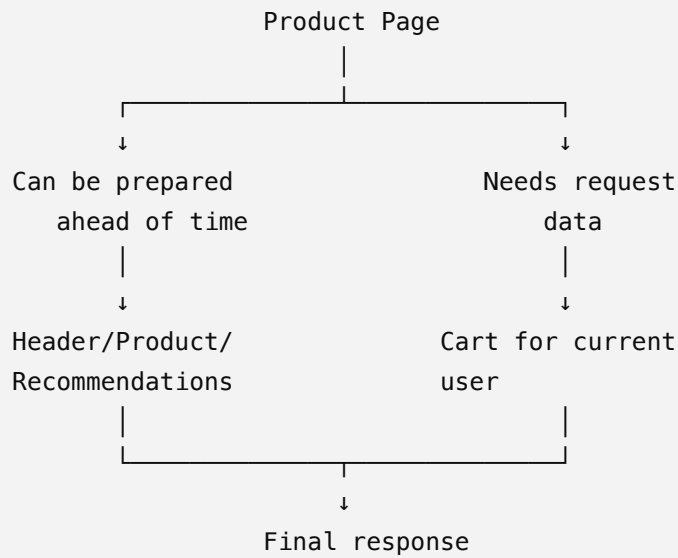
Suppose:

Component	Nature
Header	static
Product	cached
Recommendations	cached
Cart	user-specific/dynamic

Instead of saying:

"The whole page is dynamic because Cart is dynamic."

PPR lets us think:



That's the key idea. 🔥

2. Let's use a real example

Imagine Amazon-like product page:

```
export default async function ProductPage() {
  return (
    <>
      <Header />

      <Product />

      <Recommendations />

      <Suspense fallback={<CartSkeleton />}>
        <Cart />
      </Suspense>
    </>
  );
}
```

Now:

Header

same for everybody

Can be prepared ahead of time.

Product

product 123



Suppose we've cached it using:

```
"use cache";
```

It can also be prepared/cached.

Recommendations

Could also be cached.

Cart

This is different:

```
User A → 2 items  
User B → 7 items  
User C → empty
```



So Cart depends on the current request/user.

Therefore:

```
Cart = dynamic
```



3. Here's the beautiful part

The server doesn't have to wait for everything before sending something useful.

It can conceptually produce:

```
| Header  
|  
| iPhone 17  
| ₹79,999  
|  
| Recommended products  
|
```



Cart: loading...

Then later:

Cart data arrives



Cart component is streamed



Browser fills that section



So the user can start seeing useful content **before every dynamic piece has finished**.

That's where **Suspense + streaming** become important.

4. PPR does NOT mean "everything becomes SSR"

This is a very important distinction.

Suppose:

Header

Product

Recommendations

Cart



You might think:

"Cart is dynamic, therefore SSR the entire page."

That's the older coarse mental model.

With the modern approach, think:

Header



pre-renderable

Product



cached/pre-renderable

Recommendations



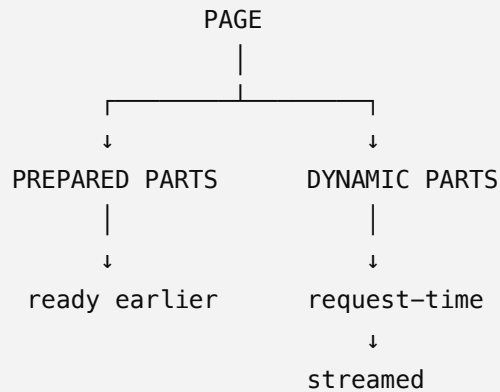
cached/pre-renderable

Cart



↓
request-time dynamic

So:



5. Where does "use cache" fit?

This is where your previous lesson connects.

Suppose:

```

async function Product() {
  "use cache";

  const product = await getProduct();

  return <div>{product.name}</div>;
}
  
```

You're saying:

"The result of this server computation can be cached."

Then:

```

"use cache"
  ↓
cache this computation
  ↓
result can be reused
  ↓
can contribute to pre-rendered content
  
```

Meanwhile:

```
async function Cart() {  
  const user = await getCurrentUser();  
  
  const cart = await getCart(user.id);  
  
  return <div>...</div>;  
}
```

This is request-dependent.

So:

Product
→ cacheable

Cart
→ dynamic



6. And where does **Suspense** fit?

This is another place people mix things up.

Suspense does **not** mean:

"This component is dynamic."



Instead, think:

"If this async work isn't ready yet, here's what I want to show meanwhile."

Example:

```
<Suspense fallback={<CartSkeleton />}>  
  <Cart />  
</Suspense>
```

The mental model:

Cart not ready
↓
show CartSkeleton
↓



```
Cart resolves
  ↓
stream Cart result
```

So:

Dynamic $\neq$ Suspense



Suspense = boundary for waiting/streaming

Dynamic = request-dependent computation

They often work together, but they're not the same concept.

7. Complete request flow

Now let's put **everything you've learned** together.

User requests:

```
GET /products/123
```



Step 1 — Request reaches Next.js

```
Browser
  ↓
CDN / network layer
  ↓
Next.js
```



Step 2 — Next.js renders the route

It encounters:

```
Header
Product
Recommendations
Cart
```



Step 3 — Cached/pre-renderable pieces

```
Header
Product
Recommendations
```



↓
reuse prepared/cached result

Step 4 — Dynamic Cart

Cart
↓
needs current request/user
↓
execute at request time



Step 5 — Suspense boundary

Cart isn't ready
↓
CartSkeleton



Step 6 — Server sends/streams result

Conceptually:

HTML / RSC response
↓
shell arrives
↓
dynamic Cart result arrives
↓
React updates the appropriate boundary



Step 7 — Browser

User sees:

Header
Product
Recommendations
Cart



without having to wait for the slowest dynamic operation before receiving the initial useful UI.



The most important mental model

Don't think:

Page = Static OR Dynamic



Think:

```
Page
├── pre-renderable/cached parts
└── request-time dynamic parts
```



And:

```
"use cache"
  ↓
makes a server computation/component cacheable

cacheLife()
  ↓
controls its cache lifecycle

cacheTag()
  ↓
labels the cached result

Suspense
  ↓
defines an async boundary

PPR
  ↓
allows pre-rendered content + dynamic portions
to coexist
```



Interview answer 🎯

If an interviewer asks "What is Partial Prerendering?", a clean answer is:

"Partial Prerendering allows a route to have a pre-rendered shell containing static or cached content while dynamic parts are resolved at request time and streamed into the page. So instead of treating the entire route as either static or dynamic, we can compose pre-renderable and dynamic parts together."

That's the answer I'd memorize. 🙌

Next: we'll go one level deeper into **exactly what gets generated/stored during PPR and what happens on the first request vs later requests**. That's the piece that makes the whole thing click.



okay

Lesson 6.4 — PPR Under the Hood: First Request vs Later Requests

Now let's make PPR **concrete**. This is the part where the abstract diagrams become an actual request lifecycle.

We'll keep using:

```
/products/123
```



with:

```
Header  
Product  
Recommendations  
Cart
```



where:

Header	→ static
Product	→ "use cache"
Recommendations	→ "use cache"
Cart	→ dynamic/user-specific



1. First, forget the word "page"

This is the biggest mental shift.

Instead of:

```
/products/123  
  ↓  
ONE page  
  ↓  
static OR dynamic
```



think:

/products/123

↓

ONE UI tree

↓



Different parts can have different rendering/cache characteristics.

2. Build/prerender time

Suppose Next.js knows that:

Product 123

can be cached.

It can execute the cacheable part ahead of time:

Product component

↓

"use cache"

↓

run server computation

↓

Product result

↓

stored as cacheable result

Same idea for:

Recommendations

So conceptually, Next.js has something reusable:

CACHE

Product 123

Recommendations
Header/static shell

⚠ Don't think of this as simply "a complete HTML page sitting in a cache."

With Cache Components, the important mental model is that **specific server-rendered/cacheable results can be reused**.

3. Now User A requests `/products/123`

Browser
|
| GET /products/123
↓
Next.js

Next.js needs:

Header
Product
Recommendations
Cart

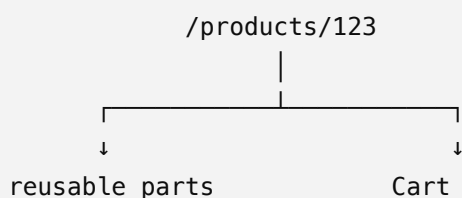
It can reuse:

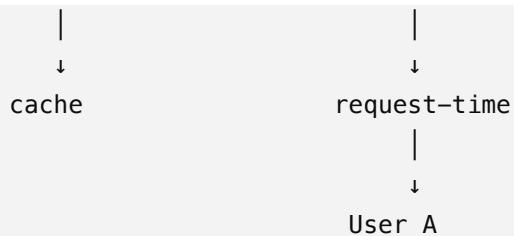
Header	→ prepared
Product	→ cached
Recommendations	→ cached

But:

Cart
↓
depends on User A
↓
must resolve for this request

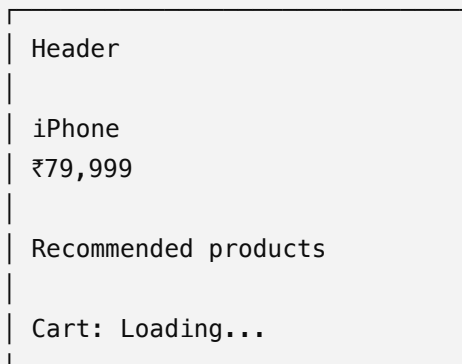
So conceptually:



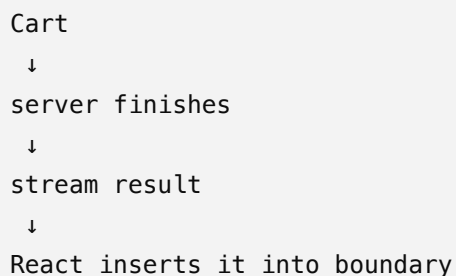


4. What does the browser receive?

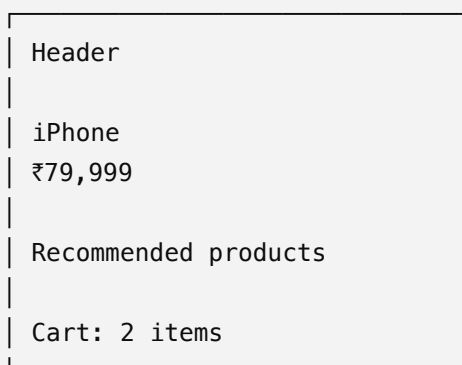
Initially, the browser can receive the prepared portion:



Then:



And the browser ends up with:



5. Now User B requests the SAME product

This is where PPR becomes really useful.

User B:

```
GET /products/123
```



The product itself doesn't depend on the user:

```
Product 123
```



So Next.js can reuse the existing cached/prepared result.

But:

```
Cart
```



is different:

```
User A → 2 items
```



```
User B → 7 items
```

Therefore:

```
Product
```

```
↓
```

```
same reusable result
```

```
Cart
```

```
↓
```

```
new request-specific result
```



The **same prepared product content** can serve many users while the dynamic portion is resolved separately.

6. Now the important question:

"Does PPR mean Next.js creates one HTML file with a hole?"

Not exactly. 🤔

That's a useful **visual analogy**, but don't take it literally.

A better mental model is:

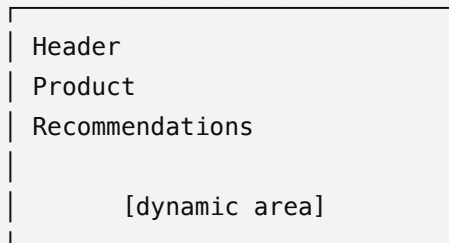
```
Pre-renderable/cacheable server output
+
request-time dynamic server output
+
React Server Component / streaming machinery
```



The "hole" is a conceptual way to understand the dynamic portion.

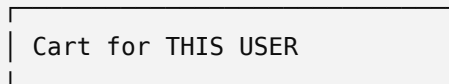
Think:

PREPARED



+

REQUEST TIME



Then they are composed for the browser.

7. Now compare this with traditional SSR

Traditional SSR mental model

Request:

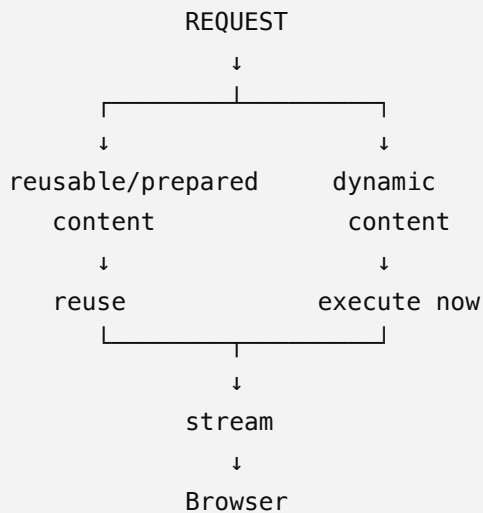
```
User
↓
Server
↓
run_page
↓
fetch_product
↓
fetch_recommendations
↓
fetch_cart
```



↓
generate response
↓
Browser

The server needs to perform the request-time work for the page.

PPR mental model



So the key advantage is **granularity**.

8. Where does caching actually happen?

This is where I want you to separate three ideas.

A. Rendering

Question:

"When does this server computation happen?"

build/prerender time
OR
request time

B. Caching

Question:

"Can I reuse the result?"

"use cache"



C. Streaming

Question:

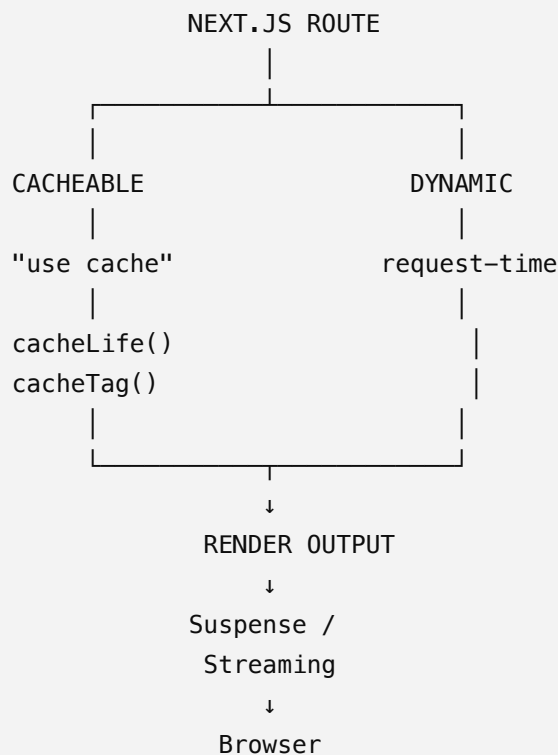
"Can I send parts as they become available?"

Suspense



These are related but **not the same thing**.

9. Your mental model should now be



10. One subtle but VERY important point

A dynamic component doesn't magically become cached just because it's inside a PPR page.

For example:

</> TypeScript



```
async function Cart() {  
  const user = await getUser();  
  
  return <div>{user.cart}</div>;  
}
```

This is still request-dependent.

PPR doesn't say:

"Everything is cached."

It says, conceptually:

"We can combine reusable/pre-renderable content with request-time dynamic content."

That's a much better interview explanation.

Interview question

Interviewer:

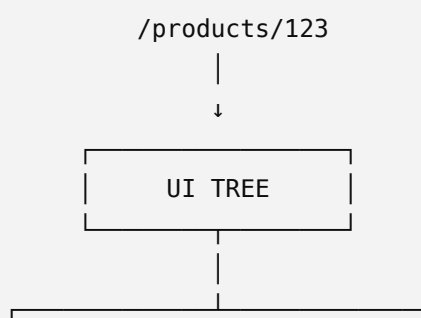
"If a page contains a dynamic user-specific component, does the entire page have to be dynamically rendered?"

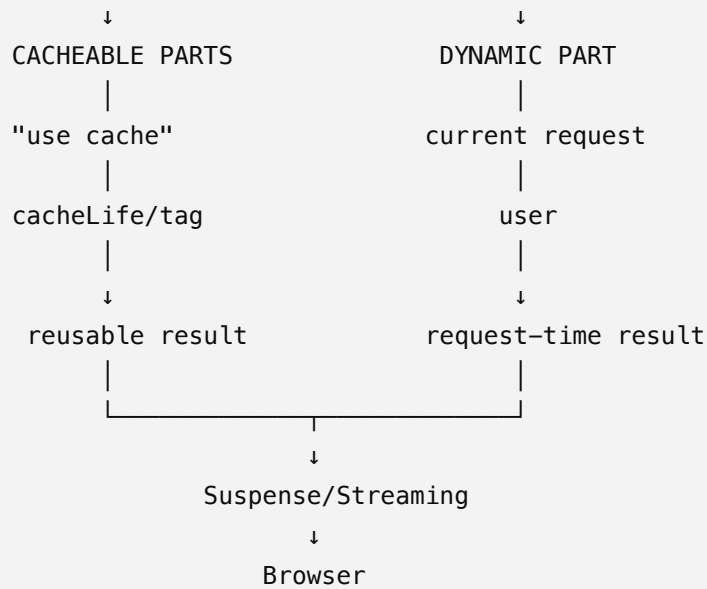
Your answer:

"Not necessarily with the modern Cache Components/PPR model. Cacheable or pre-renderable portions can be prepared and reused, while a user-specific portion can remain request-time dynamic and be streamed through a Suspense boundary."

That's a **very strong mental model**.

One final diagram to lock it in 🧠





🔥 That is PPR at the mental-model level.

Next we'll tackle the question that usually causes confusion:

"If `use cache` caches a component's result, what exactly is being cached — HTML, RSC payload, data, or the component itself?"

That's an important one because it connects **Cache Components → React Server Components → HTML → browser**.



Yes — this is the **exact confusion to clear up**. 🤔 Once you understand this, Server Components become much less magical.

1. "If Server Component code isn't in the browser, how does it show on screen?"

Because **the browser doesn't need the Server Component code to display its result**.

Think about a restaurant 🍕 :

```
Chef (server)
  ↓
uses recipe + ingredients
  ↓
prepares pizza
  ↓
waiter delivers pizza
  ↓
Customer sees pizza
```



The customer doesn't need the **recipe** or the **kitchen** to see the pizza.

Same idea:

```
SERVER
Server Component code
  ↓
executes
  ↓
produces result
  ↓
HTML / RSC payload
  ↓
BROWSER
  ↓
screen
```



2. Concrete example

You write:

```
</> TypeScript

// app/products/page.tsx

export default async function ProductsPage() {
  const products = await db.product.findMany();

  return (
    <main>
      <h1>Products</h1>

      {products.map(product => (
        <div key={product.id}>
          {product.name}
        </div>
      ))}
    </main>
  );
}
```



```
    </main>
  );
}
```

This is a **Server Component**.

The browser does NOT need:

- ✗ ProductsPage.tsx
- ✗ db.product.findMany()
- ✗ database credentials
- ✗ database connection



The server executes:

```
ProductsPage()
  ↓
DB query
  ↓
React renders result
```



Suppose the result is:

```
Products:
iPhone
MacBook
AirPods
```



The browser can receive/render markup representing that result:

 HTML



```
<main>
  <h1>Products</h1>
  <div>iPhone</div>
  <div>MacBook</div>
  <div>AirPods</div>
</main>
```

So the browser says:

"Cool, I have markup. I'll paint this on the screen."

It doesn't need to know **how the server produced it**.

3. Then what is RSC doing?

Here's the important distinction:

Server Component CODE

↓

runs on server

↓

Server Component RESULT

↓

RSC payload

↓

browser React



The browser receives the **result/representation**, not the original server implementation.

That's why:



Browser:

"Let me execute ProductsPage.tsx"

vs



Browser:

"I received the result of ProductsPage."



4. Can you SEE Router Cache in your browser? 🙄

Yes — but there's an important caveat.

You won't normally open DevTools and see:

Router Cache

/products/123

/products/456



as a nice readable cache panel.

Next.js doesn't expose Router Cache as a simple Chrome DevTools storage bucket like:

Application

├ Local Storage

├ Session Storage

├ IndexedDB

└ Router Cache ← **X** not normally here



Router Cache is an **internal client-side cache maintained by the Next.js App Router**, not ordinary `localStorage`.

5. But you CAN inspect evidence of it 🔍

This is actually a great experiment for you.

Open your Next.js app in Chrome.

Then:

Chrome DevTools



Network



Navigate using:

</> TypeScript

```
<Link href="/products/123">
```



rather than:

HTML

```
<a href="/products/123">
```



You can inspect the network requests and see requests associated with the App Router/RSC navigation.

Depending on your Next.js version and dev/prod mode, you'll see RSC-related request/response information.

You may encounter headers such as:

RSC

Next-Router-State-Tree

Next-Router-Prefetch



and responses associated with the RSC protocol.

The exact presentation varies between development/production and Next.js versions.

6. Here's a really good experiment

Make two pages:

```
/products/123  
/products/456
```



and navigate:

```
123  
↓  
456  
↓  
123
```



using:

```
</> TypeScript  
  
<Link href="/products/123">
```



Watch:

```
DevTools  
→ Network
```



You'll be able to observe the App Router making RSC/navigation requests.

Then try:

```
/products/123
```



→

```
/products/456
```



→

```
/products/123
```



The Router Cache can allow previously received route information to be reused according to its current caching behavior.

7. Can you visually see the cached Server Component?

This needs a careful distinction.

✗ You generally won't see:

Next.js Server Cache

Product component
cached result

inside Chrome DevTools.

Why?

Because that cache is **on the server**, not in your browser.

Remember:

"use cache"

↓

SERVER-SIDE CACHE

while:

Router Cache

↓

CLIENT-SIDE

They're different.

8. So where can you inspect each one?

Thing	Where it lives	Can you inspect it?
Server Component source	Server/build	Not as browser JS
"use cache" result	Server-side cache	Not directly through normal browser DevTools
RSC response	Network	✔ Yes
HTML	Network → document response	✔ Yes

Thing	Where it lives	Can you inspect it?
Client JS	Network/Sources	✅ Yes
Router Cache	Next.js client runtime	⚠️ Not as a normal DevTools storage panel
localStorage	Browser	✅ Yes
sessionStorage	Browser	✅ Yes
HTTP/browser cache	Browser DevTools	✅ Partially inspectable

9. And here's a SUPER useful experiment for you

Open:

Chrome DevTools
→ Network
→ click the document request
→ Response



You'll be able to inspect the initial HTML response.

Then navigate with `<Link>` and inspect the subsequent requests.

You'll start seeing the difference between:

INITIAL LOAD

HTML
+
RSC information
+
Client JS



and:

CLIENT NAVIGATION

RSC-related response
+
existing browser document



That distinction is really important for App Router interviews.

10. One more thing: don't confuse "showing" with "executing"

This is the key sentence:

A browser can display the output of code without possessing or executing the code that generated that output.

For example:

Server Component:

```
<h1>{product.name}</h1>
  ↓
server knows product.name = "iPhone"
  ↓
result
  ↓
<h1>iPhone</h1>
  ↓
browser displays it
```

The browser doesn't need:

`</>` TypeScript

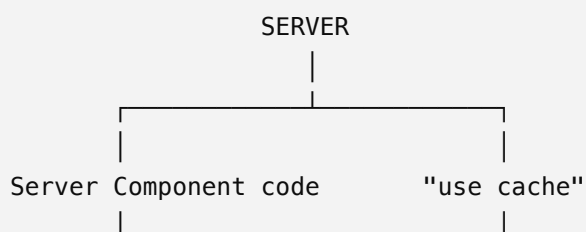
```
<h1>{product.name}</h1>
```

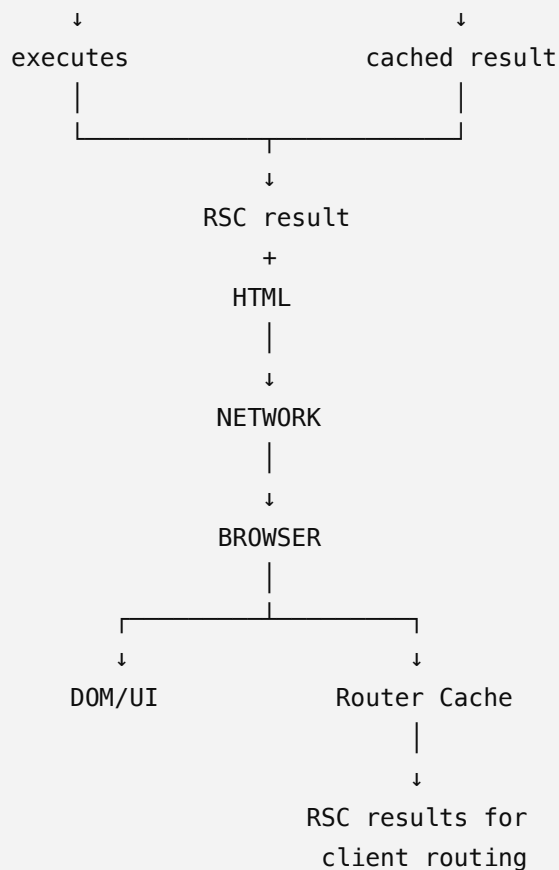
to be able to display:

iPhone

Final picture

Keep these two worlds separate:





🔥 The one thing to remember:

Server Component code stays on the server. Its *result* crosses the network.

And yes, **you can absolutely open DevTools and inspect the HTML and RSC network traffic.** You just generally can't open a little "Router Cache" folder and browse it like localStorage.

If you want, we can next do a **hands-on DevTools experiment** where I give you a tiny Next.js page and tell you **exactly what to click in Chrome DevTools and what you should expect to see.** That'll make this stick for real. 🙄



i willdo it after i build nextjs app ..next

Absolutely 🤪 We'll do the DevTools experiment **after you build the Next.js app** so you can actually see the HTML, RSC requests, navigation, and caching instead of just imagining them.

Next → Lesson 6.8 🚀

We'll move into:

Hydration vs Reconciliation vs RSC Updates

We'll answer:

Server Component renders on server

↓

HTML reaches browser

↓

Client Component JS arrives

↓

What exactly gets hydrated?

↓

What does React reconcile?

↓

What happens during router.refresh()?

↓

What happens during <Link> navigation?



And we'll use one small example throughout so the differences are crystal clear. 🧠



how any more?

Haha 🤔 you're asking **how many lessons are left in the whole Next.js roadmap**, right?

You completed **Lesson 4**, and we're currently around **Lesson 6.7** in the deeper caching/RSC section.

Remaining roadmap

Lesson	Topic	Status
6.8	Hydration vs Reconciliation vs RSC Updates	➡️ SOON Next
6.9	Full request lifecycle — putting everything together	🕒
7	Next.js Performance	🕒
8	Authentication & Authorization	🕒

Lesson	Topic	Status
9	Middleware / Proxy & request interception	🕒
10	Route Handlers & APIs	🕒
11	Error Handling & Loading UX	🕒
12	Forms, Server Actions & Mutations — interview deep dive	🕒
13	SEO, Metadata & OG Images	🕒
14	Images, Fonts & Scripts	🕒
15	Deployment, Vercel, CDN & Edge	🕒
16	Testing Next.js apps	🕒
17	Security & production best practices	🕒
18	Next.js System Design	🕒
19	Interview questions + debugging scenarios	🕒
20	Build the Web Performance Analyzer 🚀	🕒

So roughly 14 major lessons/sections remain.

But don't worry — several of those are much smaller than the caching/RSC section we've just gone through. 😂

And 6.8 → 6.9 is basically the final deep architectural piece. After that we'll move into practical/interview-focused Next.js topics.

Your journey so far

- Lesson 1 → Server vs Client Components

Lesson 2 → App Router & Routing

Lesson 3 → Navigation & Routing details

Lesson 4 → Data Fetching

Lesson 5 → Mutations & Server Actions

Lesson 6 → Caching + Rendering architecture
 - └ Cache Components
 - └ use cache
 - └ cacheLife
- ✓

✓

✓

✓

✓

🔄

✓

✓

✓



└─ cacheTag	✓
└─ PPR	✓
└─ RSC	✓
└─ 6.8 Hydration/Reconciliation	→ SOON

So we're **well past the fundamentals now.** 🔥

And after the roadmap, we'll use all of this in your **Web Performance Analyzer project**, which is where the concepts should really become second nature.



ChatGPT can make mistakes. Check important info.